

SETUHAUL LOGISTICS

Optional Add-ons for the Project

Two practical ways to make the core solution stronger

Before you start

These are optional. First make sure the basic flow works: the driver reports a delay, the system finds the correct shipment, checks the appointment, and helps the driver choose a workable slot. Add the ideas below only after that flow is stable.

1. • Add location only when it helps

A driver may say, “I should reach by 7 PM.” That is useful, but it is still only an estimate. The driver may be guessing, traffic may have changed, or the message may have been sent some time ago. The optional improvement is simple: allow the driver to share the current location once. Use that location to calculate a route ETA and compare it with what the driver said. This is not continuous tracking. It is only a location snapshot shared during the conversation.

What students are trying to achieve

Use the location to make the slot suggestion more sensible. Do not add a map feature just for display. The location should change the ETA, the arrival buffer, or the recommended slot.

A simple example

INFORMATION	VALUE
Current appointment	6:00 PM
Driver says	“I should reach by 6:30 PM.”
Location-based ETA	6:45 PM
Available option 1	7:00 PM — only 15 minutes of buffer
Available option 2	7:30 PM — 45 minutes of buffer

A useful answer would be:

Example agent response

“Based on the location you shared, your expected arrival is around 6:45 PM. The 7:00 PM slot is possible, but it leaves only 15 minutes of buffer. The 7:30 PM slot is safer. Which one would you like to choose?”

How the application can request browser location

The Python agent or backend cannot directly read the driver's browser location. The conversation should first ask for permission. When the driver agrees, the application should pause the current workflow, ask the frontend to collect one location snapshot, and resume only after the browser returns a result.

Suggested interaction pattern

1. The assistant asks: “Would you like to share your current location?”
2. If the driver agrees, return a client action such as `REQUEST_BROWSER_LOCATION` and mark the case as waiting for browser input.
3. The frontend shows a Share location button. Only the user click should trigger the browser permission request.
4. The browser returns latitude, longitude, accuracy and capture time, or a denied/error result.
5. The frontend sends the result to the backend. The saved conversation or case then continues from the same point.
6. If the driver declines or location fails, continue using the driver-declared ETA.

Design freedom: students may use their own pending-action state, a framework interrupt, or another human-in-the-loop pattern. The required outcome is that the workflow waits safely for browser input.

Illustrative chat and browser permission flow. The exact browser prompt will vary by device and browser.

Suggested routing service: Geoapify

After receiving the coordinates, the backend can call the Geoapify Routing API with the truck's current location and the destination facility coordinates. The API can return route distance and estimated travel time for truck-related routing modes. The application should calculate and store the resulting route ETA, the provider and the calculation time.

Reference: Geoapify Routing API documentation

Browser note: a deployed application normally uses `navigator.geolocation.getCurrentPosition()` after the driver clicks the button. The browser asks for explicit permission and the application generally needs to run over HTTPS. Use a one-time location snapshot, not continuous tracking.

What needs to be built

- Ask the driver whether they want to share a one-time browser location. Location sharing must remain optional.
- When the driver agrees, pause or interrupt the current workflow and return a frontend action such as `REQUEST_BROWSER_LOCATION`. The backend tool itself cannot open the browser permission prompt.

- The frontend should call the browser geolocation capability only after the driver clicks Share location, then send latitude, longitude, accuracy and capture time back to the backend.
- Use the Geoapify Routing API to calculate route distance and travel time from the shared coordinates to the destination facility. Use an appropriate truck routing mode.
- Keep the driver-declared ETA and route-based ETA separately. Do not overwrite one with the other.
- Use the ETA and arrival buffer to rank or explain slot options.
- If location is denied, unavailable or stale, continue with the normal driver-declared ETA workflow.

Data to keep

DATA	WHY IT MATTERS
Shared location and time	A location sent 30 minutes ago may no longer be useful.
Driver-declared ETA	Shows what the driver believes.
Route-based ETA	Shows what the map or route service calculated.
ETA source used for scheduling	Lets us later check whether the choice was sensible.
Actual gate-in time	Used to see which ETA was closest to reality.

Cases students should test

- The driver shares an ETA but no location.
- The driver shares location but gives no ETA.
- The driver ETA and route ETA are almost the same.
- The two ETAs differ by a large amount.
- The shared location is old.
- The location service fails or times out.
- The driver shares another location later and the ETA changes.
- The truck has already reached the gate, so actual gate-in becomes the latest truth.
- The driver does not want to share location. The normal workflow must still continue.

2. · Show whether the solution actually helped

A working demo is not enough. The final presentation should answer a few simple business questions: Did the system give a better ETA? Did the driver get a usable slot faster? Did fewer cases need a person from operations? Did the recommended slot reduce the risk of extra waiting?

The aim is not to create a large analytics platform. A few well-chosen measures are enough, as long as they come from the actual workflow.

What to measure

MEASURE	WHAT IT MEANS AND HOW TO CHECK IT
Time to resolve the case	Time from the first driver message to a confirmed or usable plan.
Human help needed	Share of cases where an operations executive had to step in.
Self-service rescheduling	Share of driver requests completed without operations taking over.
ETA error	Difference between the ETA used by the system and the actual gate-in time.
First option accepted	How often the driver accepted the first recommended slot.
Estimated waiting reduced	Expected wait under the old plan minus expected wait after the chosen slot.

A simple before-and-after view

Students can use historical data or a controlled test set. The comparison should be fair: use similar facilities, similar types of delay, and the same definition for each measure.

QUESTION	MANUAL PROCESS	WITH THE SOLUTION
How long did resolution take?	Example: 18 minutes	Example: 7 minutes
How many cases needed operations?	Example: 7 out of 10	Example: 3 out of 10
How accurate was the ETA?	Example: 29-minute error	Example: 17-minute error

The values above are only examples. Students must calculate their own results from the data they generate or process.

Where LangSmith and CloudWatch fit

TOOL	USE IT TO ANSWER
LangSmith	What did the agent understand? Which tools did it call? Did it recommend only the slots returned by the scheduling tool? How many questions did it ask?
CloudWatch or application logs	Was the application slow? Did the location service fail? How long did scheduling take? How many cases were completed, failed, or sent to a human?

Keep the setup small. A few useful traces and metrics are better than a dashboard full of numbers that do not lead to any action.

Minimum data needed for measurement

DATA TO STORE	WHY IT IS NEEDED
Case start and resolution time	Shows how long the full exception took to resolve.
Time when options were generated	Shows how long the system took to produce useful choices.
Human intervention required	Shows whether operations had to take over.
ETA source used	Shows whether the system relied on the driver, location service, or another source.
Predicted ETA and actual gate-in time	Lets the team calculate ETA error.
Old and new projected waiting time	Lets the team estimate whether the recommendation reduced waiting.

What students should show in the final demo

- One case where location clearly improves the ETA or slot choice.
- One case where location is missing or fails, but the normal workflow still works.
- The original ETA, driver ETA, route ETA, and actual gate-in time for a few shipments.
- A small comparison of resolution time and human involvement before and after the solution.
- One LangSmith trace showing the agent and tool flow, plus a small CloudWatch view or application log summary showing latency, failures, and completed cases.

The final question

Did the location feature help the system make a better recommendation, and can the team show that the overall workflow became faster, more accurate, or less dependent on manual coordination?